

Bypassing the Exascale Bottleneck

forkrun : Contention-Free, Born-Local NUMA
Parallelization

Anthony Barone

Computational Geophysicist & Systems Researcher

anthonywbarone@gmail.com • (858) 735-2342

The Data Locality Problem

Modern HPC nodes are increasingly complex:

- Multiple NUMA domains
- Multiple memory tiers
- Multiple accelerators

Yet many runtime systems still rely on:

- Reactive page migration
- Dynamic profiling
- Manual affinity tuning

Question

Can we make locality a property of the data itself rather than a property of the scheduler?

Born-Local NUMA

Core Idea

Instead of moving pages after allocation:

- Route data to the correct NUMA node before allocation
- Let Linux first-touch instantiate pages locally
- Align chunk boundaries with record boundaries
- Ensure workers consume data where it was born

Result

Locality becomes structural rather than reactive.

Solving Cold Start: Born-Local NUMA

Seekable Inputs

- Use `fstat()` forecasting
- Scale chunk size dynamically
- Ensure all NUMA domains receive work immediately

Streaming Inputs

- Backpressure-aware chunk sizing
- Geometric ramp from 64KB → 1MB
- No manual tuning required

Goal: Achieve good initial placement before runtime feedback exists.

Solving Cold Start: Delimiter Scanner

Cheap Information Scan

- pre-flight delimiter scan (SIMD Popcount - AVX2/NEON)
- estimate workload density → compute initial batch sizing
- scan until $W_{\max} * L_{\max}$ lines || EOF || 1st worker spawns

Controller enters steady state drastically faster.

Ramp Up → Steady-State

- Geometric ramp up to $L_{\max}$
- PID-like steady state
- Early flush on stall + starve

Goal: Achieve good initial batch size before total line count is known.

A Generalizable Pattern

Common Problem: Dynamic systems begin with little information.

- poor initial placement
- oscillation
- warm-up inefficiency

Common Solution:

1. Cheap initial scan (when possible) → gather information
2. Geometric ramp up → compensate for lack of initial information
3. Feedback-driven steady-state → stable equilibrium

Born-Local NUMA Requires (one of):

- Ownership signals that are available before execution
- Ownership that emerges from the placement decision itself

Born-Local vs Conventional Approaches

Approach	Primary Strength	Primary Limitation
Auto-NUMA	Fully automatic	Migration overhead, slow to adapt
Static Pinning	Excellent locality	Manual tuning, tail imbalance
Static Round Robin	Even load distribution	Ignores execution locality, tail imbalance
Traditional Work Stealing	Dynamic load balancing	Destroys NUMA affinity
SICM (ECP)	Adaptive, app-guided placement	Requires runtime observation/warm-up
Born-Local	Locality by construction	Requires pre-execution ownership signals (detected or manufactured)

Real-World Telemetry

i9-7940X (booted with `numa=fake=4`)

- Seekable files:
 - **0.0% cross-socket traffic**
- Piped streaming:
 - **<3% cross-socket steals**
- NUMA coordination overhead:
 - **~1.5–2% CPU**
- Core utilization:
 - **90–99% across all cores**

All achieved with ZERO warm-up, profiling, or manual tuning. How?

Because **resource allocation is the fixed point of a feedback process** — which simultaneously **manufactures the ownership it relies on**.

Why I Reached Out To CORSys

- **Beyond Streaming:** Streaming is just the domain where we discovered the pattern. The underlying mechanism is broader.
- **The Paradigm: Ownership-Driven Placement + Equilibrium-Driven Ownership**
 1. A discrete unit of work or data exists.
 2. We can use feedback processes to identify and/or manufacture ownership *before* execution begins.
 3. That ownership is highly predictive (perhaps by design) of future access patterns.
- **Generalizing the Principle:**
 - **forkrun:** `chunk` + *queue backpressure* → NUMA node
 - **PDE Solvers:** `mesh tile` + *halo-exchange latency* → NUMA node
 - **DBMS:** `database shard` + *lock-wait pressure* → NUMA node
 - **GPU:** `work queue` + *warp occupancy* → Accelerator

Potential Collaboration Areas

Immediate Next Step: Hardware Validation

- **Near-term need:** Validate/benchmark on real multi-socket NUMA hardware (EPYC / dual Xeon) to quantify true interconnect latency avoidance.

Academic Collaboration

- Ownership-driven placement beyond streaming runtimes
- Transitioning from reactive dynamic migration to proactive bootstrapping

The Core Research Question

- SICM solves the general placement problem dynamically. But cold starts trigger expensive page-migration storms.

Can we use proactive, ownership-driven routing + equilibrium-driven ownership to make the initial placement decision at birth, leaving SICM's tiering to handle the dynamic corrections that pre-execution placement can't anticipate?

Long-Term Vision: STTR

Phase 1

libforkrun

- reusable C runtime
- workflow integration
- Python bindings
- AI/ML/HPC data preparation

Phase 2

GPU-resident work distribution: `grun` (GPU-resident forkrun)

- extend claiming architecture to GPU execution
- investigate accelerator-side scheduling
- reduce expertise required for efficient GPU utilization

Vision: Democratized Performance

Today: Expert-level performance often requires expert-level tuning.

Goal: Scientists focus on science.

The runtime handles:

- locality
- scheduling
- work distribution
- data movement

The Bigger Questions

How much of performance engineering can be moved into the runtime itself?

How much more science can existing infrastructure produce by doing so?

Thank You

Questions & Discussion

Live benchmark running in background.

Feel free to ask about any numbers you see.

APPENDIX

Supplemental Architecture Details

Protecting the Boundary: Contention-Free Coordination

The Problem: Mutexes and CAS retry loops bounce cache lines across the interconnect — an invalidation storm collapses scaling regardless of placement quality.

Single-Slot Claiming:

- One `atomic_fetch_add()` per batch
- No CAS retries, no mutexes, no dispatcher bottleneck on the hot path

Node-Local Escrow Recovery:

- Per-node escrow pipes isolate crash recovery to the local socket
- No cross-socket cache thrashing during failure handling